

Pontifícia Universidade Católica de Minas Gerais (PUC Minas)
Laboratório de Experimentação de Software – 2026/1

Raphael Brito
Yan Cota

Laboratório 05

GraphQL vs. REST: Um Experimento Controlado

Estudo de tempo de resposta e tamanho de payload na API do GitHub

Belo Horizonte
2026

Sumário

1	Introdução	1
2	Questões de Pesquisa e Hipóteses	1
3	Objetivos	1
3.1	Objetivo Geral	1
3.2	Objetivos Específicos	1
4	Metodologia	1
4.1	Desenho do Experimento	1
4.2	Ambiente de Execução dos Trials	2
4.3	Procedimento de Coleta e Análise	2
4.4	Reprodução e Replicação	2
5	Resultados	2
5.1	RQ1 – Tempo de resposta	2
5.2	RQ2 – Tamanho do payload	3
6	Discussão dos Resultados	4
6.1	Interpretação	4
6.2	Ameaças à Validade	4
7	Conclusão e Resposta às Hipóteses	4

1 Introdução

A escolha do estilo arquitetural de uma API influencia diretamente o desempenho percebido e o consumo de recursos de uma aplicação. O modelo **REST**, predominante há mais de uma década, expõe recursos em *endpoints* fixos que retornam representações completas, frequentemente incorrendo em *over-fetching* – o transporte de campos que o cliente não utilizará. O **GraphQL**, por sua vez, permite ao cliente declarar exatamente quais campos deseja, prometendo respostas mais enxutas.

Neste laboratório conduzimos um **experimento controlado** comparando os dois paradigmas na API pública do GitHub, tendo como objeto experimental o repositório do motor de xadrez de código aberto `official-stockfish/Stockfish`. Duas variáveis de resposta foram medidas em cada requisição: a **latência** (tempo de ida e volta, em milissegundos) e o **tamanho do payload** (em bytes). Ao todo foram consolidadas **487 medições válidas** (HTTP 200), distribuídas entre os quatro tipos de recurso avaliados.

A motivação central é verificar empiricamente se a vantagem conceitual do GraphQL – evitar o *over-fetching* – se traduz em ganhos estatisticamente significativos de *banda* e de *tempo*, e em que medida tais ganhos dependem do recurso consultado.

2 Questões de Pesquisa e Hipóteses

O experimento responde a duas questões de pesquisa:

RQ1 – Tempo de resposta As respostas às consultas GraphQL são mais rápidas (menor latência) do que às consultas REST?

RQ2 – Tamanho do payload As respostas às consultas GraphQL possuem tamanho menor (em bytes) do que às consultas REST?

Para cada questão formulamos hipóteses nula (H_0) e alternativa (H_1), testadas de forma **unilateral** (esperamos que o GraphQL seja *menor* em ambas as métricas):

RQ1 (latência).

$$H_{0,1} : \text{Latência}_{\text{GraphQL}} \geq \text{Latência}_{\text{REST}}$$

$$H_{1,1} : \text{Latência}_{\text{GraphQL}} < \text{Latência}_{\text{REST}}$$

RQ2 (tamanho).

$$H_{0,2} : \text{Tamanho}_{\text{GraphQL}} \geq \text{Tamanho}_{\text{REST}}$$

$$H_{1,2} : \text{Tamanho}_{\text{GraphQL}} < \text{Tamanho}_{\text{REST}}$$

Nossa expectativa inicial é que $H_{1,2}$ seja fortemente confirmada (o GraphQL transfere apenas os campos solicitados), enquanto $H_{1,1}$ seja apenas *parcialmente* confirmada, pois a latência depende também de fatores de rede e de processamento do servidor que não favorecem intrinsecamente nenhum dos paradigmas.

3 Objetivos

3.1 Objetivo Geral

Avaliar quantitativamente, sob rigor estatístico, os impactos de desempenho (latência) e de eficiência de rede (tamanho de payload) decorrentes da escolha entre GraphQL e REST na API do GitHub.

3.2 Objetivos Específicos

- Comparar as distribuições de latência e de tamanho entre os dois paradigmas, por recurso, com o teste de Mann-Whitney U unilateral.
- Quantificar a magnitude prática das diferenças por meio do tamanho de efeito *Cliff's Delta*.
- Mensurar a economia de banda absoluta (bytes trafegados) ao longo de todo o experimento.
- Documentar o procedimento de forma a permitir reprodução e replicação integrais.

4 Metodologia

4.1 Desenho do Experimento

Adotou-se um projeto **dentro de sujeitos** (*within-subject*), em que o mesmo objeto experimental é submetido aos dois tratamentos. As **variáveis independentes** são: (i) o *paradigma de API* (REST ou GraphQL) e (ii) o *tipo de recurso* consultado, com quatro níveis – **repo** (metadados do repositório), **commits**, **issues** e **pulls** (listas dos 30 itens mais recentes). As **variáveis dependentes** são a latência (Y_1 , em ms) e o tamanho do payload (Y_2 , em bytes).

Tratamentos. O tratamento REST consiste em requisições HTTP GET aos *endpoints* da API REST v3. O tratamento GraphQL consiste em requisições HTTP POST à API GraphQL v4, cujas *queries* solicitam estritamente o mesmo conjunto lógico de campos que seria exibido em uma interface, evitando o *over-fetching*.

Randomização e intercalação. Para isolar fatores de confusão (oscilações de rede, flutuações temporais do servidor e *cache* de rota), todas as requisições de

uma execução são embaralhadas (`random.shuffle`) e **intercaladas**, de modo que REST e GraphQL nunca sejam medidos em rajadas separadas.

Tamanho amostral. O desenho prevê $N \geq 30$ medições por combinação (tratamento $\times$ recurso), satisfazendo o Teorema do Limite Central. Nesta execução consolidada obtivemos **487 medições válidas** (todas com status HTTP 200), entre 60 e 62 por célula, sendo 243 requisições REST e 244 GraphQL.

4.2 Ambiente de Execução dos Trials

Para permitir reprodução, registramos o ambiente em que os *trials* foram realizados:

- **Linguagem/runtime:** Python 3.8+.
- **Bibliotecas:** `requests` (cliente HTTP), `scipy` (testes estatísticos), `pandas` (processamento), `matplotlib/seaborn` (gráficos) e `python-dotenv` (configuração).
- **Medição de latência:** `time.perf_counter()` envolvendo a chamada, medindo o RTT completo do lado do cliente.
- **Medição de tamanho:** `len(response.content)`, isto é, o número de bytes do corpo da resposta.
- **Autenticação:** *Personal Access Token* do GitHub (cabeçalho `Authorization: Bearer`), com `Cache-Control: no-cache` e *User-Agent* dedicado.
- **Servidores:** infraestrutura de produção do GitHub (`api.github.com`), acessada por uma máquina cliente sob conexão de internet pública (banda larga).
- **Atraso entre chamadas:** 1,5s, evitando *rate limiting* e rajadas consecutivas.
- **Janela de coleta:** 26 de junho de 2026 (*timestamps* registrados em UTC).

4.3 Procedimento de Coleta e Análise

O *pipeline* é composto por quatro etapas, cada uma em um *script* dedicado no diretório `code/`:

1. `collector.py` – executa as requisições intercaladas e grava cada medição bem-sucedida em `data/raw_data.csv`.
2. `processor.py` – consolida estatísticas descritivas por célula.

3. `analyzer.py` – aplica os testes de hipótese e gera o relatório estatístico.
4. `visualizer.py` – gera os gráficos comparativos.

A análise estatística é **não-paramétrica**, justificada pelo teste de normalidade de **Shapiro-Wilk**, que rejeitou a normalidade ($p < 0,001$) em todas as distribuições. Assim, empregamos:

- **Teste U de Mann-Whitney** (unilateral, H_1 : GraphQL < REST), robusto a *outliers*, ao nível de significância $\alpha = 0,05$.
- **Cliff's Delta** (δ) como tamanho de efeito, interpretado como desprezível ($|\delta| < 0,147$), pequeno ($< 0,33$), médio ($< 0,474$) ou grande ($\geq 0,474$).

A notação de significância adotada nas tabelas é: $p < 0,001$ (***), $p < 0,01$ (**), $p < 0,05$ (*) e $p \geq 0,05$ (ns).

4.4 Reprodução e Replicação

O experimento é integralmente reproduzível com os comandos: `pip install -r requirements.txt`; cópia de `.env.example` para `.env` com um `GITHUB_TOKEN` válido; e execução sequencial de `collector.py`, `processor.py`, `analyzer.py` e `visualizer.py`. Os dados brutos, o relatório estatístico e os gráficos ficam versionados em `code/data/` e `code/plots/`.

5 Resultados

A Tabela 1 apresenta as medianas globais (todos os recursos agregados) de cada métrica, por paradigma.

Table 1: Medianas globais por paradigma (todos os recursos).

Métrica	GraphQL	REST
Latência (ms)	534,79	615,32
Tamanho (bytes)	—	—
Tráfego total	1,67 MB	35,04 MB

No total, o GraphQL trafegou **1,67 MB** contra **35,04 MB** do REST – uma **economia de banda de 95,2%** ao longo do experimento.

5.1 RQ1 – Tempo de resposta

A Tabela 2 resume o teste de Mann-Whitney U para a latência. O resultado é **misto**: o GraphQL é significativamente mais rápido em *issues* e *pulls*, mas *não* há evidência de ganho em *repo* e *commits*.

Table 2: RQ1 – Mann-Whitney U (unilateral, GraphQL < REST) para latência.

Recurso	M. GQL	M. REST	δ	Sig.
repo	503,2	595,0	-0,062	ns
commits	635,0	621,1	-0,125	ns
issues	626,9	746,9	-0,262	**
pulls	581,7	853,6	-0,407	***

M. = mediana (ms); δ = Cliff's Delta. ns: $p \geq 0,05$; **: $p < 0,01$; ***: $p < 0,001$.

A Figura 1 mostra os *boxplots* de latência por recurso. As distribuições são fortemente assimétricas e com *outliers*, o que reforça a escolha por testes não-paramétricos. Onde o payload REST é muito volumoso (*pulls*, *issues*), o tempo de transferência também cresce, e o GraphQL leva vantagem; em recursos pequenos (*repo*) ou de custo de processamento semelhante (*commits*), a diferença desaparece.

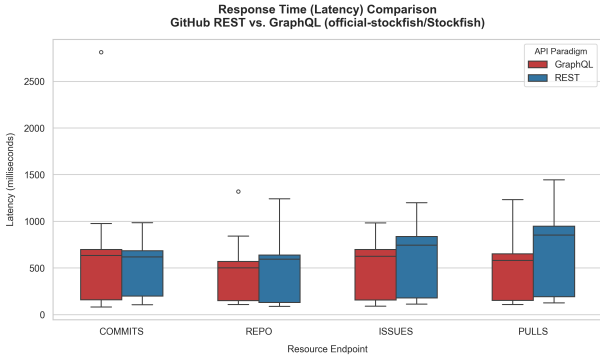


Figure 1: Distribuição da latência (ms) por recurso e paradigma. GraphQL é significativamente mais rápido em *pulls* (***) e *issues* (**); sem diferença significativa em *repo* e *commits*.

- **repo** — sem diferença significativa ($p = 0,279$; $\delta = -0,062$, desprezível).
- **commits** — sem diferença significativa ($p = 0,117$; $\delta = -0,125$, desprezível).
- **issues** — GraphQL mais rápido ($p = 0,0066$; $\delta = -0,262$, pequeno).
- **pulls** — GraphQL mais rápido ($p = 5,68 \times 10^{-5}$; $\delta = -0,407$, médio).

5.2 RQ2 – Tamanho do payload

A Tabela 3 apresenta o teste para o tamanho do payload. O resultado é **unânime e contundente**: em todos os recursos o GraphQL retorna payloads significativamente menores, com tamanho de efeito **máximo** ($\delta = -1,000$, ou seja, *todas* as respostas GraphQL são menores que *todas* as REST) e $p < 10^{-21}$.

Table 3: RQ2 – Mann-Whitney U (unilateral, GraphQL < REST) para tamanho.

Recurso	M. GQL	M. REST	Red.
repo	249 B	6,9 KB	96,4 %
commits	33,2 KB	126,5 KB	73,8 %
issues	5,0 KB	115,7 KB	95,7 %
pulls	3,8 KB	529,1 KB	99,3 %

M. = mediana; Red. = redução da mediana (GraphQL vs. REST). Em todos os casos: $\delta = -1,000$ (grande) e $p < 0,001$ (***)

As Figuras 2 e 3 comparam o tamanho mediano em escalas linear e logarítmica. Na escala linear, as barras do GraphQL são quase imperceptíveis frente ao REST – evidência visual da diferença de *ordem de grandeza*, tornada legível pela escala logarítmica.

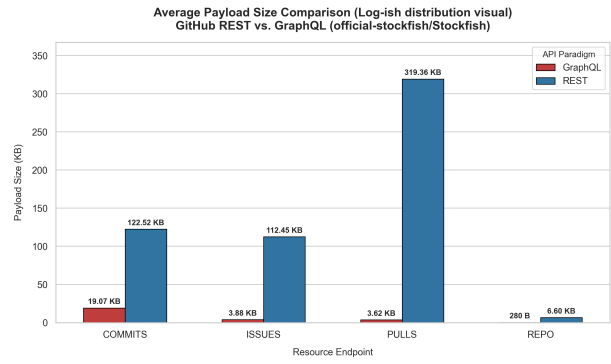


Figure 2: Tamanho mediano do payload por recurso (escala linear). A barra do GraphQL é praticamente nula diante do REST.

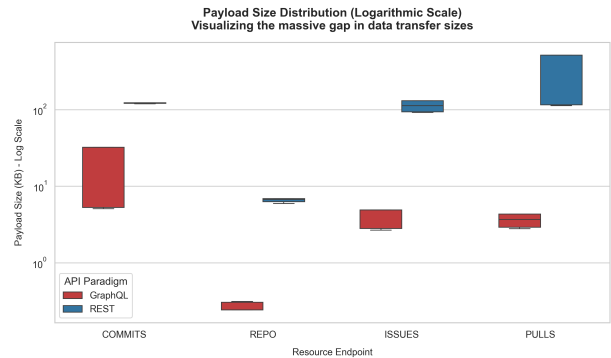


Figure 3: Tamanho mediano do payload por recurso (escala logarítmica), evidenciando a diferença de ordem de grandeza entre os paradigmas.

- **repo** — redução de 96,4% (249 B vs. 6,9 KB; ***).
- **commits** — redução de 73,8% (33,2 KB vs. 126,5 KB; ***).
- **issues** — redução de 95,7% (5,0 KB vs. 115,7 KB; ***).

- **pulls** — redução de 99,3 % (3,8 KB vs. 529,1 KB; ***).

6 Discussão dos Resultados

6.1 Interpretação

- **O ganho de banda é a vantagem dominante.** A redução do payload é consistente, de grande efeito e estatisticamente inquestionável em todos os recursos. A economia agregada de 95,2 % confirma que a prevenção de *over-fetching* é o principal benefício prático do GraphQL.
- **O ganho de latência é real, porém condicional.** O GraphQL só foi mais rápido onde a resposta REST é volumosa (**pulls**, **issues**), sugerindo que parte do ganho de tempo decorre do *menor volume transferido*, não de processamento intrinsecamente mais rápido. Em recursos pequenos, o custo fixo de rede domina e anula a vantagem.
- **Magnitude de efeito acompanha o volume.** O Cliff's Delta da latência cresce em módulo de **repo** (−0,062) a **pulls** (−0,407), na mesma ordem em que cresce a diferença de tamanho dos payloads – um indício de causalidade entre volume e tempo.

6.2 Ameaças à Validade

- **Validade interna:** *jitter* de rede e *cache* de CDN podem inflar latências. Mitigamos com randomização total, intercalação, cabeçalho **no-cache** e atraso entre chamadas.
- **Validade de construto:** a comparação só é justa se ambas as APIs retornam o *mesmo* conteúdo lógico; por isso as *queries* GraphQL foram desenhadas para espelhar os campos exibidos pelo REST.
- **Validade externa:** os resultados refletem a infraestrutura de larga escala do GitHub sob internet pública e não se generalizam automaticamente a APIs locais ou de outras arquiteturas.

7 Conclusão e Resposta às Hipóteses

O experimento, com 487 medições válidas sobre o repositório `official-stockfish/Stockfish`, permite responder às questões de pesquisa:

- **RQ1 (latência):** hipótese $H_{1,1}$ **parcialmente confirmada**. Rejeitamos $H_{0,1}$ em **issues** e **pulls** (GraphQL mais rápido), mas não em **repo** e **commits**.
- **RQ2 (tamanho):** hipótese $H_{1,2}$ **totalmente confirmada**. Rejeitamos $H_{0,2}$ em todos os recursos, com efeito máximo ($\delta = -1,000$) e economia agregada de 95,2 %.

A Tabela 4 sintetiza o desfecho de cada hipótese.

Table 4: Síntese da avaliação das hipóteses.

Hip.	Descrição resumida	Status
$H_{1,1}$	GraphQL apresenta menor latência que REST.	Parcial
$H_{1,2}$	GraphQL apresenta payload menor que REST.	Confirmada

Recomendação. Para aplicações sensíveis a banda – clientes móveis, redes limitadas ou cenários de alto volume de requisições – a adoção do GraphQL é fortemente recomendada pelo ganho expressivo de payload. Já o ganho de *tempo* deve ser esperado sobretudo em consultas a recursos de resposta volumosa; em recursos pequenos, a diferença de latência tende a ser irrelevante.